



**Port Said University
Faculty of Management
Technology & Information
Systems**



Structured Programming

البرمجة الهيكلية

Prepared By

**Technology & Information Systems
Department**

Prof. Dr. Mohamed Eisa

Contents

Unit 1	Basic Concepts	2
Unit 2	Control Structures and decisions.	26
Unit 3	Functions Basics. Advanced Features of Functions	66
Unit 4	Single Dimension Arrays Single Dimension Arrays (Sort and Search) Multidimensional Arrays	81
Unit 5	Files and C-Strings	92
Unit 6	Pointers	97
Unit 7	Introduction to Object – Oriented Programming	106

Basic Concepts

We will write and understand the **first program in C++** programming. We are writing a simple C++ program that prints “Hello World!” message. Let's see the program first and then we will discuss each and every part of it in detail.

Hello World Program in C++

```
/*  
  
* Multiple line  
  
* comment  
  
*/  
  
#include<iostream>  
  
//Single line comment  
  
using namespace std;  
  
//This is where the execution of program begins  
  
int main()  
  
{  
  
    // displays Hello World! on screen  
  
    cout<<"Hello World!";  
  
    return 0;  
  
}
```

Output:

Hello World!

Let's discuss each and every part of the above program.

1. Comments – You can see two types of comments in the above program

```
// This is a single line comment
```

```
/* This is a multiple line comment
```

```
* suitable for long comments
```

```
*/
```

Comments as the names suggests are just a text written by programmer during code development. Comment doesn't affect your program logic in any way, you can write whatever you want in comments but it should be related to the code and have some meaning so that when someone else look into your code, the person should understand what you did in the code by just reading your comment.

For example:

```
/* This function adds two integer numbers
```

```
* and returns the result as an integer value
```

```
*/
```

```
int sum(int num1, int num2) {
```

```
    return num1+num2;
```

```
}
```

Now if someone reads my comment he or she can understand what I did there just by reading my comment. This improves readability of your code and when you are working on a project with your team mates, this becomes essential aspect.

2. #include<iostream> – This statements tells the compiler to include iostream file. This file contains pre defined input/output functions that we can use in our program.

3. using namespace std; – A namespace is like a region, where we have functions, variables etc and their scope is limited to that particular region. Here std is a namespace name, this tells the compiler to look into that particular region for all the variables, functions, etc. I will not discuss this in detail here as it may confuse you.

4. int main() – As the name suggests this is the main function of our program and the execution of program begins with this function, the int here is the return type which indicates to the compiler that this function will return a integer value. That is the main reason we have a return 0 statement at the end of main function.

5. cout << “Hello World!”; – The cout object belongs to the iostream file and the purpose of this object is to display the content between double quotes as it is on the screen. This object can also display the value of variables on screen

6. return 0; – This statement returns value 0 from the main() function which indicates that the execution of main function is successful. The value 1 represents failed execution.

Variables in C++

A variable is a name which is associated with a value that can be changed. For example when I write `int num=20`; here variable name is num which is associated with value 20, int is a data type that represents that this variable can hold integer values. We will cover the data types next. We will discuss about variables.

Syntax of declaring a variable in C++

```
data_type variable1_name = value1, variable2_name = value2;
```

For example:

```
int num1=20, num2=100;
```

We can also write it like this:

```
int num1,num2;
```

```
num1=20;
```

```
num2=100;
```

Types of variables

Variables can be categorized based on their data type. For example, in the above example we have seen integer types variables. Following are the types of variables available in C++.

int: These type of variables holds integer value.

char: holds character value like 'c', 'F', 'B', 'p', 'q' etc.

bool: holds boolean value true or false.

double: double-precision floating point value.

float: Single-precision floating point value.

Types of variables based on their scope

Before going further let's discuss what scope is first. We have seen the curly braces in the program like this:

```
int main {  
  
    //Some code  
  
}
```

Any variable declared inside these curly braces have scope limited within these curly braces, if you declare a variable in main() function and try to use that variable outside main() function then you will get compilation error.

Now that we have understood what is scope. Let's move on to the types of variables based on the scope.

1.Globalvariable

2. Local variable

Global Variable

A variable declared outside of any function (including main as well) is called global variable. Global variables have their scope throughout the program, they can be accessed anywhere in the program, in the main, in the user defined function, anywhere.

Let's take an example to understand it:

Global variable example

Here we have a global variable myVar, that is declared outside of main. We have accessed the variable twice in the main() function without any issues.

```
#include <iostream>

using namespace std;

// This is a global variable

char myVar = 'A';

int main()

{

    cout <<"Value of myVar: "<< myVar<<endl;

    myVar='Z';

    cout <<"Value of myVar: "<< myVar;

    return 0;

}
```

Output:

Value of myVar: A

Value of myVar: Z

Local variable

Local variables are declared inside the braces of any user defined function, main function, loops or any control statements (if, if-else etc) and have their scope limited inside those braces.

Local variable example

```
#include <iostream>

using namespace std;

char myFuncn() {

// This is a local variable

char myVar = 'A';

}

int main()

{

    cout <<"Value of myVar: "<< myVar<<endl;

    myVar='Z';

    cout <<"Value of myVar: "<< myVar;

    return 0;

}
```

Output:

Compile time error, because we are trying to access the variable myVar outside of its

scope. The scope of myVar is limited to the body of function myFuncn(), inside those braces.

Can global and local variable have same name in C++

Lets see an example having same name global and local variable.

```
#include <iostream>

using namespace std;

// This is a global variable

char myVar = 'A';

char myFuncn() {

    // This is a local variable

    char myVar = 'B';

    return myVar;

}

int main()

{

    cout << "Funcn call: " << myFuncn() << endl;

    cout << "Value of myVar: " << myVar << endl;

    myVar='Z';

    cout << "Funcn call: " << myFuncn() << endl;

    cout << "Value of myVar: " << myVar << endl;

    return 0;
```

```
}
```

Output:

Funcn call: B

Value of myVar: A

Funcn call: B

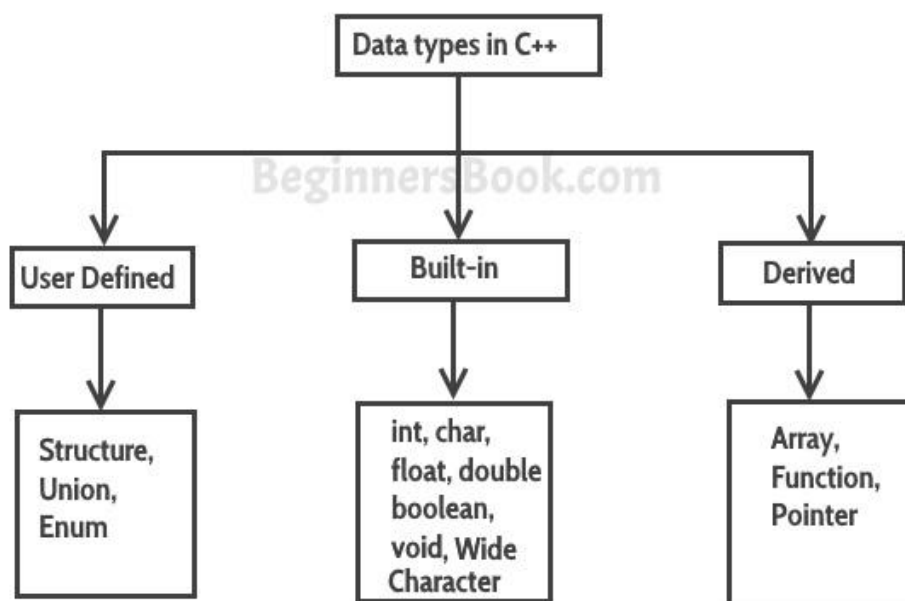
Value of myVar: Z

As you can see that when I changed the value of myVar in the main function, it only changed the value of global variable myVar because local variable myVar scope is limited to the function myFuncn().

Data Types in C++

Data types define the type of data a variable can hold, for example an integer variable can hold integer data, a character type variable can hold character data etc.

Data types in C++ are categorised in three groups: **Built-in**, **user-defined** and **Derived**.



Built in data types

char: For characters. Size 1 byte.

```
char ch = 'A';
```

int: For integers. Size 2 bytes.

```
int num = 100;
```

float: For single precision floating point. Size 4 bytes.

```
float num = 123.78987;
```

double: For double precision floating point. Size 8 bytes.

```
double num = 10098.98899;
```

bool: For booleans, true or false.

```
bool b = true;
```

wchar_t: Wide Character. This should be avoided because its size is implementation defined and not reliable.

User-defined data types

We have three types of user-defined data types in C++

1.struct

2.union

3. enum

I have covered them in details. For now just remember that these comes under user-defined data types.

Derived data types in C++

We have three types of derived-defined data types in C++

1.Array

2.Function

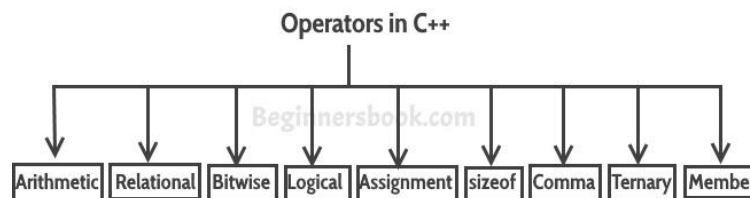
3. Pointer

They are wide topics of C++ and I have covered them in separate section.

Operators in C++

Operator represents an action. For example + is an operator that represents addition. An operator works on two or more operands and produce an output. For example 3+4+5 here + operator works on three operands and produce 12 as output.

Types of Operators in C++



- 1) Basic Arithmetic Operators
- 2) Assignment Operators
- 3) Auto-increment and Auto-decrement Operators
- 4) Logical Operators
- 5) Comparison (relational) operators
- 6) Bitwise Operator
- 7) Ternary Operator

1) Basic Arithmetic Operators

Basic arithmetic operators are: +, -, *, /, %

+ is for addition.

- is for subtraction.

* is for multiplication.

/ is for division.

% is for modulo

Note: Modulo operator returns remainder, for example 20 % 5 would return 0

Example of Arithmetic Operators

```
#include <iostream>

using namespace std;

int main(){

    int num1 = 240;

    int num2 = 40;

    cout<<"num1 + num2: "<<(num1 + num2)<<endl;

    cout<<"num1 - num2: "<<(num1 - num2)<<endl;

    cout<<"num1 * num2: "<<(num1 * num2)<<endl;

    cout<<"num1 / num2: "<<(num1 / num2)<<endl;

    cout<<"num1 % num2: "<<(num1 % num2)<<endl;

    return 0;

}
```

Output:

num1 + num2: 280

num1 - num2: 200

num1 * num2: 9600

num1 / num2: 6

num1 % num2: 0

2) Assignment Operators

Assignments operators in C++ are: =, +=, -=, *=, /=, %=

num2 = num1 would assign value of variable num1 to the variable.

num2+=num1 is equal to num2 = num2+num1

num2-=num1 is equal to num2 = num2-num1

num2*=num1 is equal to num2 = num2*num1

num2/=num1 is equal to num2 = num2/num1

num2%=num1 is equal to num2 = num2%num1

Example of Assignment Operators

```
#include <iostream>
```

```
using namespace std;
```

```
int main(){
```

```
    int num1 = 240;
```

```
    int num2 = 40;
```

```
    num2 = num1;
```

```
    cout<<"= Output: "<<num2<<endl;
```

```
    num2 += num1;
```

```
    cout<<"+= Output: "<<num2<<endl;
```

```
    num2 -= num1;
```

```
    cout<<"-= Output: "<<num2<<endl;
```

```

num2 *= num1;

cout<<"*= Output: "<<num2<<endl;

num2 /= num1;

cout<<"/= Output: "<<num2<<endl;

num2 %= num1;

cout<<"%= Output: "<<num2<<endl;

return 0;

}

```

Output:

```

= Output: 240

+= Output: 480

-= Output: 240

*= Output: 57600

/= Output: 240

%= Output: 0

```

3) Auto-increment and Auto-decrement Operators

++ and --

num++ is equivalent to num=num+1;

num-- is equivalent to num=num-1;

Example of Auto-increment and Auto-decrement Operators

```
#include <iostream>
```



```

using namespace std;

int main(){

    int num1 = 240;

    int num2 = 40;

    num1++; num2--;

    cout<<"num1++ is: "<<num1<<endl;

    cout<<"num2-- is: "<<num2;

    return 0;

}

```

Output:

num1++ is: 241

num2-- is: 39

4) Logical Operators

Logical Operators are used with binary variables. They are mainly used in conditional statements and loops for evaluating a condition.

Logical operators in C++ are: &&, ||, !

Let's say we have two boolean variables b1 and b2.

b1&&b2 will return true if both b1 and b2 are true else it would return false.

b1||b2 will return false if both b1 and b2 are false else it would return true.

!b1 would return the opposite of b1, that means it would be true if b1 is false and it would return false if b1 is true.

Example of Logical Operators

```
#include <iostream>

using namespace std;

int main(){

    bool b1 = true;

    bool b2 = false;

    cout<<"b1 && b2: "<<(b1&&b2)<<endl;

    cout<<"b1 || b2: "<<(b1||b2)<<endl;

    cout<<"!(b1 && b2): "<<!(b1&&b2);

    return 0;

}
```

Output:

b1 && b2: 0

b1 || b2: 1

!(b1 && b2): 1

5) Relational operators

We have six relational operators in C++: ==, !=, >, <, >=, <=

== returns true if both the left side and right side are equal

!= returns true if left side is not equal to the right side of operator.

> returns true if left side is greater than right.

< returns true if left side is less than right side.

>= returns true if left side is greater than or equal to right side.

<= returns true if left side is less than or equal to right side.

Example of Relational operators

```
#include <iostream>
```

```
using namespace std;
```

```
int main(){
```

```
    int num1 = 240;
```

```
    int num2 =40;
```

```
    if (num1==num2) {
```

```
        cout<<"num1 and num2 are equal"<<endl;
```

```
    }
```

```
    else{
```

```
        cout<<"num1 and num2 are not equal"<<endl;
```

```
    }
```

```
    if( num1 != num2 ){
```

```
        cout<<"num1 and num2 are not equal"<<endl;
```

```
    }
```

```
    else{
```

```
        cout<<"num1 and num2 are equal"<<endl;
```

```
    }
```

```
    if( num1 > num2 ){
```

```

    cout<<"num1 is greater than num2"<<endl;

}

else{

    cout<<"num1 is not greater than num2"<<endl;

}

if( num1 >= num2 ){

    cout<<"num1 is greater than or equal to num2"<<endl;

}

else{

    cout<<"num1 is less than num2"<<endl;

}

if( num1 < num2 ){

    cout<<"num1 is less than num2"<<endl;

}

else{

    cout<<"num1 is not less than num2"<<endl;

}

if( num1 <= num2 ){

    cout<<"num1 is less than or equal to num2"<<endl;

}

else{

```

```

    cout<<"num1 is greater than num2"<<endl;

}

return 0;

}

```

Output:

num1 and num2 are not equal

num1 and num2 are not equal

num1 is greater than num2

num1 is greater than or equal to num2

num1 is not less than num2

num1 is greater than num2

6) Bitwise Operators

There are six bitwise Operators: &, |, ^, ~, <<, >>

num1 = 11; /* equal to 00001011*/

num2 = 22; /* equal to 00010110 */

Bitwise operator performs bit by bit processing.

num1 & num2 compares corresponding bits of num1 and num2 and generates 1 if both bits are equal, else it returns 0. In our case it would return: 2 which is 00000010 because in the binary form of num1 and num2 only second last bits are matching.

num1 | num2 compares corresponding bits of num1 and num2 and generates 1 if either bit is 1, else it returns 0. In our case it would return 31 which is 00011111

num1 ^ **num2** compares corresponding bits of num1 and num2 and generates 1 if they are not equal, else it returns 0. In our example it would return 29 which is equivalent to 00011101

~num1 is a complement operator that just changes the bit from 0 to 1 and 1 to 0. In our example it would return -12 which is signed 8 bit equivalent to 11110100

num1 << 2 is left shift operator that moves the bits to the left, discards the far left bit, and assigns the rightmost bit a value of 0. In our case output is 44 which is equivalent to 00101100

Note: In the example below we are providing 2 at the right side of this shift operator that is the reason bits are moving two places to the left side. We can change this number and bits would be moved by the number of bits specified on the right side of the operator. Same applies to the right side operator.

num1 >> 2 is right shift operator that moves the bits to the right, discards the far right bit, and assigns the leftmost bit a value of 0. In our case output is 2 which is equivalent to 00000010

Example of Bitwise Operators

```
#include <iostream>

using namespace std;

int main(){

    int num1 = 11; /* 11 = 00001011 */

    int num2 = 22; /* 22 = 00010110 */

    int result = 0;

    result = num1 & num2;

    cout<<"num1 & num2: "<<result<<endl;
```

```

result = num1 | num2;

cout<<"num1 | num2: "<<result<<endl;

result = num1 ^ num2;

cout<<"num1 ^ num2: "<<result<<endl;

result = ~num1;

cout<<"~num1: "<<result<<endl;

result = num1 << 2;

cout<<"num1 << 2: "<<result<<endl;

result = num1 >> 2;

cout<<"num1 >> 2: "<<result;

return 0;

}

```

Output:

num1 & num2: 2

num1 | num2: 31

num1 ^ num2: 29

~num1: -12

num1 << 2: 44 num1 >> 2: 2

7) Ternary Operator

This operator evaluates a boolean expression and assign the value based on the result.

Syntax:

variable num1 = (expression) ? value if true : value if false

If the expression results true then the first value before the colon (:) is assigned to the variable num1 else the second value is assigned to the num1.

Example of Ternary Operator

```
#include <iostream>
```

```
using namespace std;
```

```
int main(){
```

```
    int num1, num2; num1 = 99;
```

```
    /* num1 is not equal to 10 that's why
```

```
    * the second value after colon is assigned
```

```
    * to the variable num2
```

```
    */
```

```
    num2 = (num1 == 10) ? 100: 200;
```

```
    cout<<"num2: "<<num2<<endl;
```

```
    /* num1 is equal to 99 that's why
```

```
    * the first value is assigned
```

```
    * to the variable num2
```

```
    */
```

```
    num2 = (num1 == 99) ? 100: 200;
```

```
    cout<<"num2: "<<num2;
```

```
    return 0;
```



```
}
```

Output:

```
num2: 200
```

```
num2: 100
```

Miscellaneous Operators

There are few other operators in C++ such as **Comma operator** and **size of operator**.

Operator Precedence in C++

This determines which operator needs to be evaluated first if an expression has more than one operator. Operator with higher precedence at the top and lower precedence at the bottom.

Unary Operators

```
++ -- ! ~
```

Multiplicative

```
* / %
```

Additive

```
+ -
```

Shift

```
<< >> >>>
```

Relational

```
> >= < <=
```

Equality

```
== !=
```

Bitwise AND

&

Bitwise XOR

^

Bitwise OR

|

Logical AND

&&

Logical OR

||

Ternary

?:

Assignment

= += -= *= /= %= > >= < <= &= ^= |=

Control Structures

If else Statement

Sometimes we need to execute a block of statements only when a particular condition is met or not met. This is called **decision making**, as we are executing a certain code after making a decision in the program logic. For decision making in C++, we have four types of control statements (or control structures), which are as follows:

- a) if statement
- b) nested if statement
- c) if-else statement
- d) if-else-if statement

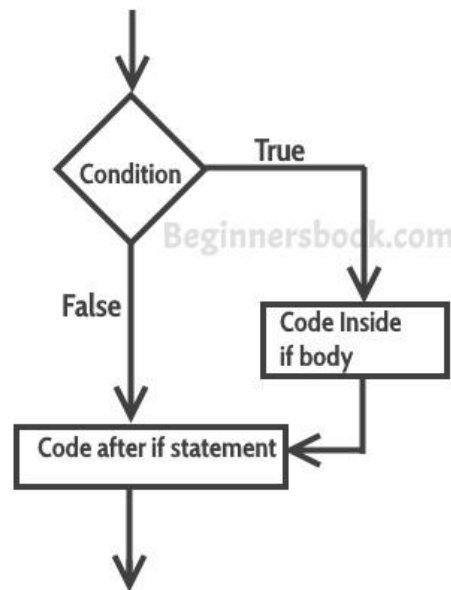
If statement in C++

If statement consists a condition, followed by statement or a set of statements as shown below:

```
If (condition){  
    Statement(s);  
}
```

The statements inside **if** parenthesis (usually referred as if body) gets executed only when the given condition is true. If the condition is false then the statements inside if body are completely ignored.

Flow diagram of If statement



Example of if statement

```
#include <iostream>
```

```
using namespace std;
```

```
int main(){
```

```
    int num=70;
```

```
    if( num < 100 ){
```

```
        /* This cout statement will only execute,
```

```
        * if the above condition is true
```

```
        */
```

```
        cout<<"number is less than 100";
```

```
    }
```

```
    if(num > 100){
```

```
        /* This cout statement will only execute,
```

```

    * if the above condition is true

    */

    cout<<"number is greater than 100";

}

return 0;

}

```

Output:

number is less than 100

Nested if statement in C++

When there is an if statement inside another if statement then it is called the **nested if statement**.

The structure of nested if looks like this:

```

if(condition_1) {

    Statement1(s);

    if(condition_2) {

        Statement2(s);

    }

}

```

Statement1 would execute if the condition_1 is true. Statement2 would only execute if both the conditions(condition_1 and condition_2) are true.

Example of Nested if statement

```

#include <iostream>

using namespace std;

int main(){

    int num=90;

    /* Nested if statement. An if statement
    * inside another if body
    */

    if( num < 100 ){

        cout<<"number is less than 100"<<endl;

        if(num > 50){

            cout<<"number is greater than 50";

        }

    }

    return 0;

}

```

Output:

number is less than 100

number is greater than 50

If else statement in C++

Sometimes you have a condition and you want to execute a block of code if condition is true and execute another piece of code if the same condition is false. This can be achieved in C++ using if-else statement.

This is how an if-else statement looks:

```
If (condition) {
```

```
    Statement(s);
```

```
}
```

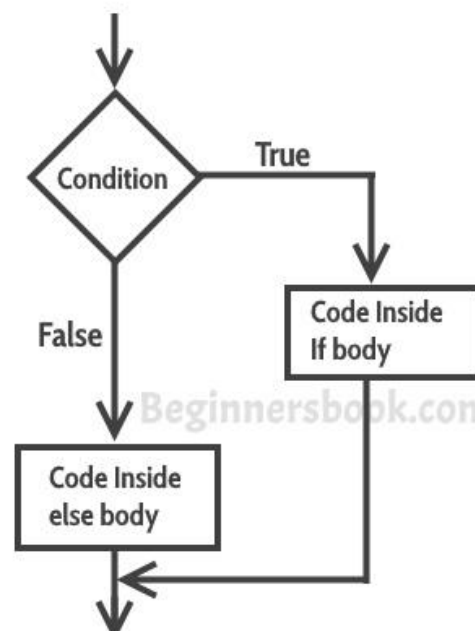
```
else {
```

```
    Statement(s);
```

```
}
```

The statements inside “if” would execute if the condition is true, and the statements inside “else” would execute if the condition is false.

Flow diagram of if-else



Example of if-else statement

```
#include <iostream>

using namespace std;

int main(){

    int num=66;

    if( num < 50 ){

        //This would run if above condition is true

        cout<<"num is less than 50";

    }

    else {

        //This would run if above condition is false

        cout<<"num is greater than or equal 50";

    }

    return 0;

}
```

Output:

num is greater than or equal 50

if-else-if Statement in C++

if-else-if statement is used when we need to check multiple conditions. In this control structure we have only one “if” and one “else”, however we can have multiple “else if” blocks. This is how it looks:


```

if(condition_1) {

    /*if condition_1 is true execute this*/

    statement(s);

}

else if(condition_2) {

    /* execute this if condition_1 is not met and

    * condition_2 is met

    */

    statement(s);

}

else if(condition_3) {

    /* execute this if condition_1 & condition_2 are

    * not met and condition_3 is met

    */

    statement(s);

}

.

.

.

else {

    /* if none of the condition is true

```

```

    * then these statements gets executed

    */

    statement(s);

}

```

Note: The most important point to note here is that in if-else-if, as soon as the condition is met, the corresponding set of statements get executed, rest gets ignored. If none of the condition is met then the statements inside “else” gets executed.

Example of if-else-if

```

#include <iostream>

using namespace std;

int main(){

    int num;

    cout<<"Enter an integer number between 1 & 99999: ";

    cin>>num;

    if(num <100 && num>=1) {

        cout<<"Its a two digit number";

    }

    else if(num <1000 && num>=100) {

        cout<<"Its a three digit number";

    }

    else if(num <10000 && num>=1000) {

```

```
    cout<<"Its a four digit number";  
  
}  
  
else if(num <100000 && num>=10000) {  
  
    cout<<"Its a five digit number";  
  
}  
  
else {  
  
    cout<<"number is not between 1 & 99999";  
  
}  
  
return 0;  
  
}
```

Output:

Enter an integer number between 1 & 99999: 8976

Its a four digit number

Switch Case statement

Switch case statement is used when we have multiple conditions and we need to perform different action based on the condition. When we have multiple conditions and we need to execute a block of statements when a particular condition is satisfied. In such case either we can use lengthy if-else-if statement or switch case. The problem with lengthy if else if is that it becomes complex when we have several conditions. The switch case is a clean and efficient method of handling such scenarios.

The **syntax of Switch case** statement:

switch (variable or an integer expression)

```
{  
  
    case constant:  
  
        //C++ code  
  
        ;  
  
    case constant:  
  
        //C++ code  
  
        ;  
  
    default:  
  
        //C++ code  
  
        ;  
  
}
```

Switch Case statement is mostly used with break statement even though the break statement is optional. We will first see an example without break statement and then we will discuss switch case with break

Example of Switch Case

```
#include <iostream>

using namespace std;

int main(){

    int num=5;

    switch(num+2) {

        case 1:

            cout<<"Case1: Value is: "<<num<<endl;

        case 2:

            cout<<"Case2: Value is: "<<num<<endl;

        case 3:

            cout<<"Case3: Value is: "<<num<<endl;

        default:

            cout<<"Default: Value is: "<<num<<endl;

    }

    return 0;

}
```

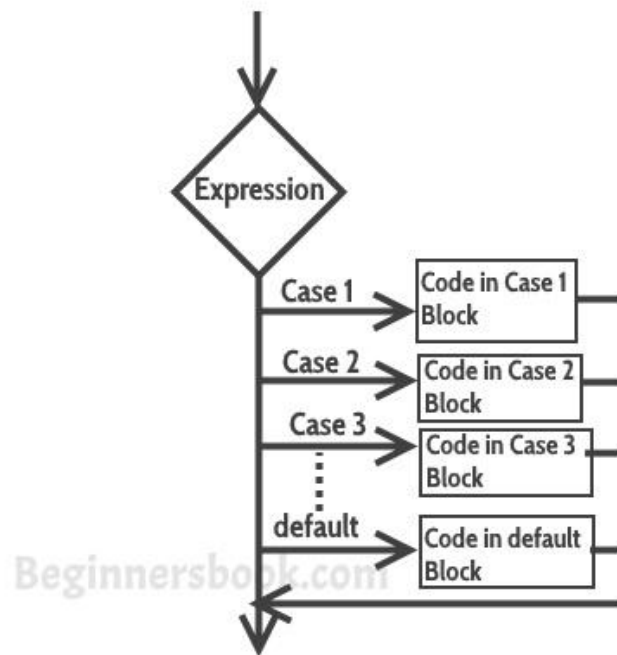
Output:

Default: Value is: 5

Explanation: In switch I gave an expression, you can give variable as well. I gave the expression $\text{num}+2$, where num value is 5 and after addition the expression resulted 7. Since there is no case defined with value 4 the default case got executed.

Switch Case Flow Diagram

It evaluates the value of expression or variable (based on whatever is given inside switch braces), then based on the outcome it executes the corresponding case.



Break statement in Switch Case

Before we discuss about break statement, let's see what happens when we don't use break statement in switch case. See the example below:

```
#include <iostream>
```

```
using namespace std;
```

```
int main(){
```

```
    int i=2;
```

```
    switch(i) {
```

```
case 1: cout<<"Case1 "<<endl;

case 2: cout<<"Case2 "<<endl;

case 3: cout<<"Case3 "<<endl;

case 4: cout<<"Case4 "<<endl;

default: cout<<"Default "<<endl;

}

return 0;

}
```

Output:

Case2

Case3

Case4

Default

In the above program, we have the variable `i` inside switch braces, which means whatever the value of variable `i` is, the corresponding case block gets executed. We have passed integer value 2 to the switch, so the control switched to the case 2, however we don't have break statement after the case 2 that caused the flow to continue to the subsequent cases till the end. However this is not what we wanted, we wanted to execute the right case block and ignore rest blocks. The solution to this issue is to use the break statement in after every case block.

Break statements are used when you want your program-flow to come out of the switch body. Whenever a break statement is encountered in the switch body, the execution flow would directly come out of the switch, ignoring rest of the cases. This is why you must end each case block with the break statement.

Let's take the same example but this time with break statement.

```
#include <iostream>

using namespace std;

int main(){

    int i=2;

    switch(i) {

        case 1:

            cout<<"Case1 "<<endl;

            break;

        case 2:

            cout<<"Case2 "<<endl;

            break;

        case 3:

            cout<<"Case3 "<<endl;

            break;

        case 4:

            cout<<"Case4 "<<endl;

            break;

        default:

            cout<<"Default "<<endl;

    }
```



```
    return 0;
}
```

Output:

Case2

Now you can see that only case 2 got executed, rest of the subsequent cases were ignored.

Why didn't I use break statement after default?

The control would itself come out of the switch after default so I didn't use break statement after it, however if you want you can use it, there is no harm in doing that.

Important Notes

1) Case doesn't always need to have order 1, 2, 3 and so on. It can have any integer value after case keyword. Also, case doesn't need to be in an ascending order always, you can specify them in any order based on the requirement.

2) You can also use characters in switch case. for example –

```
#include <iostream>
```

```
using namespace std;
```

```
int main(){
```

```
    char ch='b';
```

```
    switch(ch) {
```

```
        case 'd': cout<<"Case1 ";
```

```
        break;
```

```
        case 'b': cout<<"Case2 ";
```

```
        break;
```

```
    case 'x': cout<<"Case3 ";  
  
    break;  
  
    case 'y': cout<<"Case4 ";  
  
    break;  
  
    default: cout<<"Default ";  
  
}  
  
return 0;  
  
}
```

3) Nesting of switch statements are allowed, which means you can have switch statements inside another switch. However nested switch statements should be avoided as it makes program more complex and less readable.

For loop

A loop is used for executing a block of statements repeatedly until a particular condition is satisfied. For example, when you are displaying number from 1 to 100 you may want set the value of a variable to 1 and display it 100 times, increasing its value by 1 on each loop iteration.

In C++ we have three types of basic loops: for, while and do-while. We will learn how to use “for loop” in C++.

Syntax of for loop

```
for(initialization; condition ; increment/decrement)
```

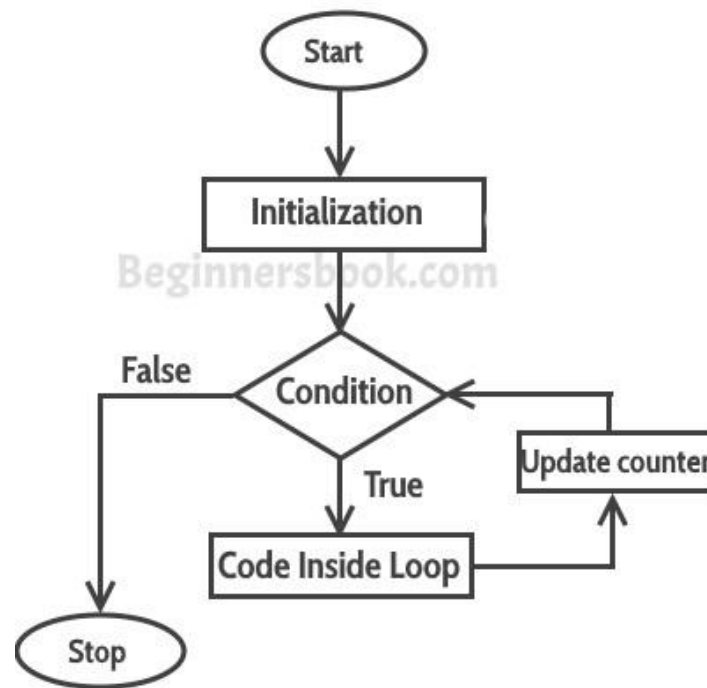
```
{
```

```
    C++ statement(s);
```

```
}
```

Flow of Execution of the for Loop

As a program executes, the interpreter always keeps track of which statement is about to be executed. We call this the control flow, or the flow of execution of the program.



First step: In for loop, initialization happens first and only once, which means that the initialization part of for loop only executes once.

Second step: Condition in for loop is evaluated on each loop iteration, if the condition is true then the statements inside for loop body gets executed. Once the condition returns false, the statements in for loop does not execute and the control gets transferred to the next statement in the program after for loop.

Third step: After every execution of for loop's body, the increment/decrement part of for loop executes that updates the loop counter.

Fourth step: After third step, the control jumps to second step and condition is re-evaluated.

The steps from second to fourth repeats until the loop condition returns false.

Example of a Simple For loop in C++

Here in the loop initialization part I have set the value of variable *i* to 1, condition is $i \leq 6$ and on each loop iteration the value of *i* increments by 1.

```
#include <iostream>

using namespace std;

int main(){

    for(int i=1; i<=6; i++){

        /* This statement would be executed

        * repeatedly until the condition

        * i<=6 returns false.

        */

        cout<<"Value of variable i is: "<<i<<endl;

    }

    return 0;

}
```

Output:

Value of variable i is: 1

Value of variable i is: 2

Value of variable i is: 3

Value of variable i is: 4

Value of variable i is: 5

Value of variable i is: 6

Infinite for loop in C++

A loop is said to be infinite when it executes repeatedly and never stops. This usually happens by mistake. When you set the condition in for loop in such a way that it never return false, it becomes infinite loop.

For example:

```
#include <iostream>

using namespace std;

int main(){

    for(int i=1; i>=1; i++){

        cout<<"Value of variable i is: "<<i<<endl;

    }

    return 0;

}
```

This is an infinite loop as we are incrementing the value of i so it would always satisfy the condition $i \geq 1$, the condition would never return false.

Here is another example of infinite for loop:

```
// infinite loop

for ( ; ; ) {

    // statement(s)

}
```

Example: display elements of array using for loop

```
#include <iostream>

using namespace std;
```

```
int main(){  
  
    int arr[]={21,9,56,99, 202};  
  
    /* We have set the value of variable i  
  
    * to 0 as the array index starts with 0  
  
    * which means the first element of array  
  
    * starts with zero index.  
  
    */  
  
    for(int i=0; i<5; i++){  
  
        cout<<arr[i]<<endl;  
  
    }  
  
    return 0;  
  
}
```

Output:

21

9

56

99

202

While loop

We discussed for loop. we will discuss while loop. As discussed earlier, loops are used for executing a block of program statements repeatedly until the given loop condition returns false.

Syntax of while loop

```
while(condition)
```

```
{
```

```
    statement(s);
```

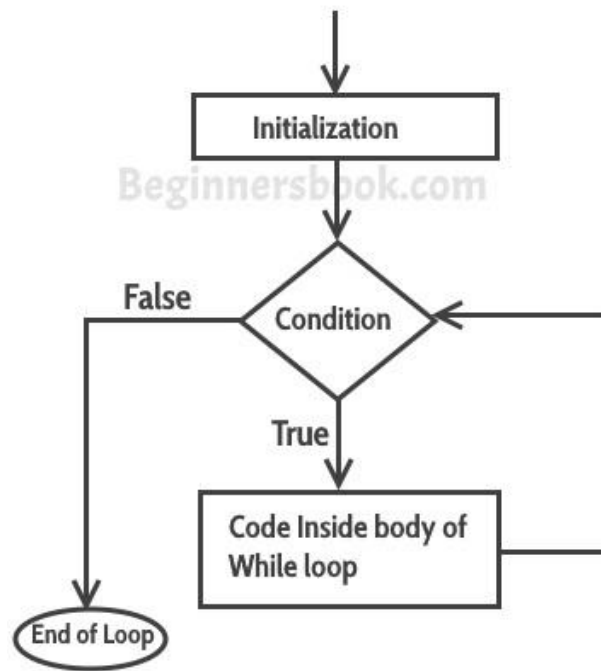
```
}
```

How while Loop works?

In while loop, condition is evaluated first and if it returns true then the statements inside while loop execute, this happens repeatedly until the condition returns false. When condition returns false, the control comes out of loop and jumps to the next statement in the program after while loop.

Note: The important point to note when using while loop is that we need to use increment or decrement statement inside while loop so that the loop variable gets changed on each iteration, and at some point condition returns false. This way we can end the execution of while loop otherwise the loop would execute indefinitely.

Flow Diagram of While loop



While Loop example in C++

```
#include <iostream>

using namespace std;

int main(){

    int i=1;

    /* The loop would continue to print

    * the value of i until the given condition

    * i<=6 returns false.

    */

    while(i<=6){

        cout<<"Value of variable i is: "<<i<<endl; i++;

    }

}
```

Output:

Value of variable i is: 1

Value of variable i is: 2

Value of variable i is: 3

Value of variable i is: 4

Value of variable i is: 5

Value of variable i is: 6

Infinite While loop

A while loop that never stops is said to be the infinite while loop, when we give the condition in such a way so that it never returns false, then the loops becomes infinite and repeats itself indefinitely.

An example of infinite while loop:

This loop would never end as I'm decrementing the value of i which is 1 so the condition $i \leq 6$ would never return false.

```
#include <iostream>
```

```
using namespace std;
```

```
int main(){
```

```
    int i=1; while(i<=6) {
```

```
        cout<<"Value of variable i is: "<<i<<endl; i--;
```

```
    }
```

```
}
```

Example: Displaying the elements of array using while loop

```
#include <iostream>

using namespace std;

int main(){

    int arr[]={21,87,15,99, -12};

    /* The array index starts with 0, the
    * first element of array has 0 index
    * and represented as arr[0]
    */

    int i=0;

    while(i<5){

        cout<<arr[i]<<endl;

        i++;

    }

}
```

Output:

21
87
15
99
-12

Do - while loop

As discussed about while loop, a loop is used for repeating a block of statements until the given loop condition returns false. We will see do-while loop. do-while loop is similar to while loop, however there is a difference between them: In while loop, condition is evaluated first and then the statements inside loop body gets executed, on the other hand in do-while loop, statements inside do-while gets executed first and then the condition is evaluated.

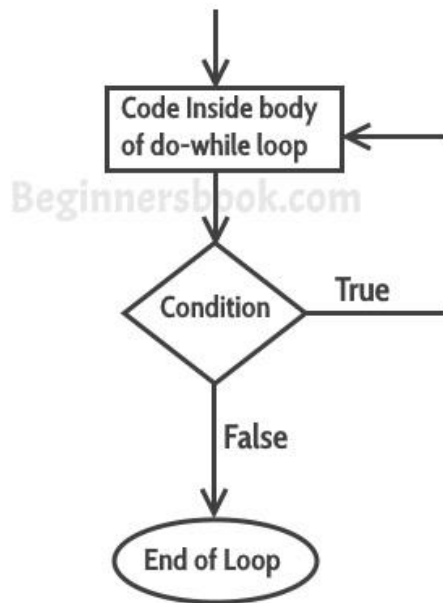
Syntax of do-while loop

```
do  
  
{  
  
    statement(s);  
  
} while(condition);
```

How do-while loop works?

First, the statements inside loop execute and then the condition gets evaluated, if the condition returns true then the control jumps to the “do” for further repeated execution of it, this happens repeatedly until the condition returns false. Once condition returns false

control jumps to the next statement in the program after do-while.



do-while loop example in C++

```
#include <iostream>

using namespace std;

int main(){

    int num=1;

    do{

        cout<<"Value of num: "<<num<<endl;

        num++;

    }while(num<=6);

    return 0;

}
```

Output:

Value of num: 1

Value of num: 2

Value of num: 3

Value of num: 4

Value of num: 5

Value of num: 6

Example: Displaying array elements using do-while loop

Here we have an integer array which has four elements. We are displaying the elements of it using do-while loop.

```
#include <iostream>

using namespace std;

int main(){

    int arr[]={21,99,15,109};

    /* Array index starts with 0, which
    * means the first element of array
    * is at index 0, arr[0]
    */

    int i=0;

    do{

        cout<<arr[i]<<endl;

        i++;

    }while(i<4);
```

```
    return 0;  
}
```

Output:

21

99

15

109

Continue Statement

Continue statement is used inside loops. Whenever a continue statement is encountered inside a loop, control directly jumps to the beginning of the loop for next iteration, skipping the execution of statements inside loop's body for the current iteration.

Syntax of continue statement

```
continue;
```

Example: continue statement inside for loop

As you can see that the output is missing the value 3, however the for loop iterate though the num value 0 to 6. This is because we have set a condition inside loop in such a way, that the continue statement is encountered when the num value is equal to 3. So for this iteration the loop skipped the cout statement and started the next iteration of loop.

```
#include <iostream>
```

```
using namespace std;
```

```
int main(){
```

```
    for (int num=0; num<=6; num++) {
```

```
        /* This means that when the value of
```

```
        * num is equal to 3 this continue statement
```

```
        * would be encountered, which would make the
```

```
        * control to jump to the beginning of loop for
```

```
        * next iteration, skipping the current iteration
```

```
        */
```



```

if (num==3) {

    continue;

}

cout<<num<<" ";

}

return 0;

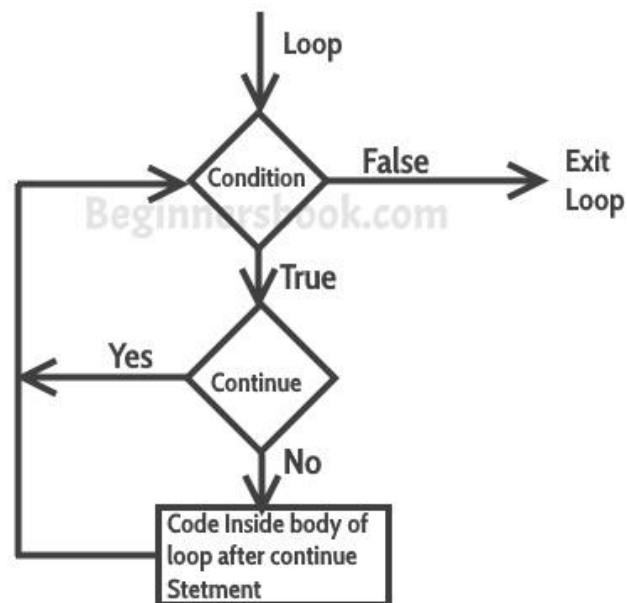
}

```

Output:

0 1 2 4 5 6

Flow Diagram of Continue Statement



Example: Use of continue in While loop

```

#include <iostream>

using namespace std;

```

```

int main(){

    int j=6;

    while (j >=0) {

        if (j==4) {

            j--;

            continue;

        }

        cout<<"Value of j: "<<j<<endl;

        j--;

    }

    return 0;

}

```

Output:

Value of j: 6

Value of j: 5

Value of j: 3

Value of j: 2

Value of j: 1

Value of j: 0

Example of continue in do-While loop

```
#include <iostream>
```

```
using namespace std;

int main(){

    int j=4;

    do {

        if (j==7) {

            j++;

            continue;

        }

        cout<<"j is: "<<j<<endl;

        j++;

    }while(j<10);

    return 0;

}
```

Output:

j is: 4

j is: 5

j is: 6

j is: 8

j is: 9

Break statement

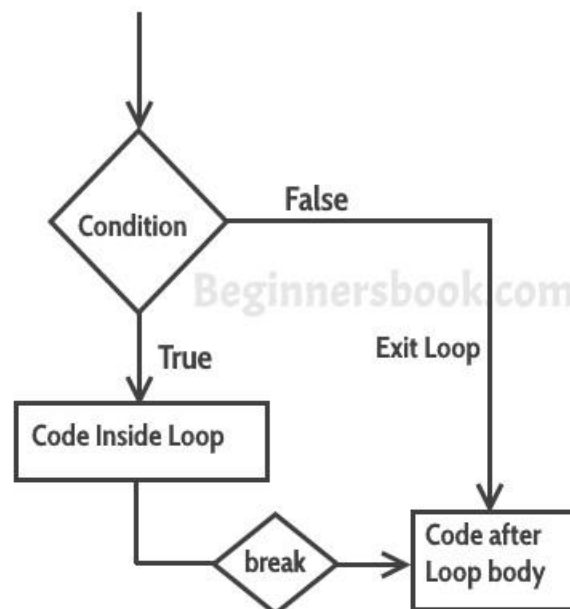
The **break statement** is used in following two scenarios:

- a) Use break statement to come out of the loop instantly. Whenever a break statement is encountered inside a loop, the control directly comes out of loop terminating it. It is used along with if statement, whenever used inside loop (see the example below) so that it occurs only for a particular condition.
- b) It is used in switch case control structure after the case blocks. Generally all cases in switch case are followed by a break statement to avoid the subsequent cases (see the example below) execution. Whenever it is encountered in switch-case block, the control comes out of the switch-case body.

Syntax of break statement

```
break;
```

break statement flow diagram



Example – Use of break statement in a while loop

In the example below, we have a while loop running from 10 to 200 but since we have a break statement that gets encountered when the loop counter variable value reaches 12, the loop gets terminated and the control jumps to the next statement in program after the loop body.

```
#include <iostream>

using namespace std;

int main(){

    int num =10;

    while(num<=200) {

        cout<<"Value of num is: "<<num<<endl;

        if (num==12) {

            break;

        }

        num++;

    }

    cout<<"Hey, I'm out of the loop";

    return 0;

}
```

Output:

Value of num is: 10

Value of num is: 11

Value of num is: 12

Hey, I'm out of the loop

Example: break statement in for loop

```
#include <iostream>

using namespace std;

int main(){

    int var;

    for (var =200; var>=10; var --) {

        cout<<"var: "<<var<<endl;

        if (var==197) {

            break;

        }

    }

    cout<<"Hey, I'm out of the loop";

    return 0;

}
```

Output:

var: 200

var: 199

var: 198

var: 197

Hey, I'm out of the loop

Example: break statement in Switch Case

```
#include <iostream>

using namespace std;

int main(){

    int num=2;

    switch (num) {

        case 1: cout<<"Case 1 "<<endl;

            break;

        case 2: cout<<"Case 2 "<<endl;

            break;

        case 3: cout<<"Case 3 "<<endl;

            break;

        default: cout<<"Default "<<endl;

    }

    cout<<"Hey, I'm out of the switch case";

    return 0;

}
```

Output:

Case 2

Hey, I'm out of the switch case

In this example, we have break statement after each Case block, this is because if we don't have it then the subsequent case block would also execute. The output of the same program without break would be:

Case 2

Case 3

Default

Hey, I'm out of the switch case

goto statement in C++ with example

The goto statement is used for transferring the control of a program to a given label. The syntax of goto statement looks like this:

```
goto label_name;
```

Program structure:

```
label1:
```

```
...
```

```
...
```

```
goto label2;
```

```
...
```

```
..
```

```
label2:
```

```
...
```


In a program we have any number of goto and label statements, the goto statement is followed by a label name, whenever goto statement is encountered, the control of the program jumps to the label specified in the goto statement.

goto statements are almost never used in any development as they are complex and makes your program much less readable and more error prone. In place of goto, you can use continue and break statement.

Example of goto statement in C++

```
#include <iostream>

using namespace std;

int main(){

    int num; cout<<"Enter a number: "; cin>>num;

    if (num % 2==0){

        goto print;

    }

    else {

        cout<<"Odd Number";

    }

    print:

    cout<<"Even Number";

    return 0;

}
```

Output:

Enter a number: 42

Even Number

Functions in C++

A function is block of code which is used to perform a particular task, for example let's say you are writing a large C++ program and in that program you want to do a particular task several number of times, like displaying value from 1 to 10, in order to do that you have to write few lines of code and you need to repeat these lines every time you display values. Another way of doing this is that you write these lines inside a function and call that function every time you want to display values. This would make you code simple, readable and reusable.

Syntax of Function

```
return_type function_name (parameter_list)

{

    //C++ Statements

}
```

Let's take a simple example to understand this concept.

A simple function example

```
#include <iostream>

using namespace std;

/* This function adds two integer values
 * and returns the result
 */int

sum(int num1, int num2){

    int num3 = num1+num2; return num3;

}
```

```
int main(){  
  
    //Calling the function  
  
    cout<<sum(1,99);  
  
    return 0;  
  
}
```

Output:

100

The same program can be written like this: Well, I am writing this program to let you understand an important term regarding functions, which is function declaration. Lets see the program first and then at the end of it we will discuss function declaration, definition and calling of function.

```
#include <iostream>  
  
using namespace std;  
  
//Function declaration  
  
int sum(int,int);  
  
//Main function  
  
int main(){  
  
    //Calling the function  
  
    cout<<sum(1,99);  
  
    return 0;  
  
}  
  
/* Function is defined after the main method
```

```

*/

int sum(int num1, int num2){

    int num3 = num1+num2;

    return num3;

}

```

Function Declaration: You have seen that I have written the same program in two ways, in the first program I didn't have any function declaration and in the second program I have function declaration at the beginning of the program. The thing is that when you define the function before the main() function in your program then you don't need to do function declaration but if you are writing your function after the main() function like we did in the second program then you need to declare the function first, else you will get compilation error.

syntax of function declaration:

```
return_type function_name(parameter_list);
```

Note: While providing parameter_list you can avoid the parameter names, just like I did in the above example. I have given int sum(int,int); instead of int sum(int num1,int num2);.

Function definition: Writing the full body of function is known as defining a function.

syntax of function definition:

```
return_type function_name(parameter_list) {

    //Statements inside function

}

```

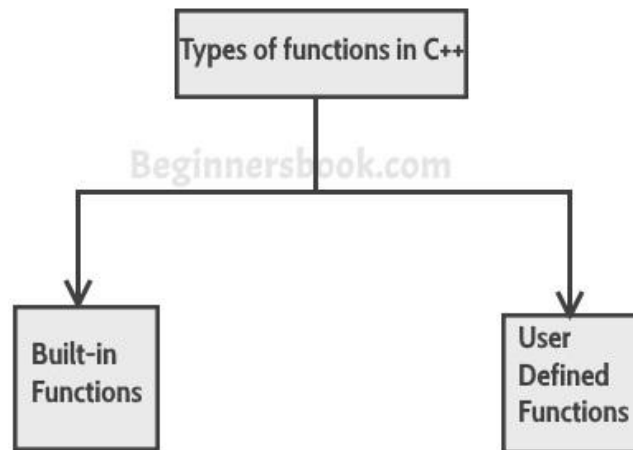
Calling function: We can call the function like this:

```
function_name(parameters);
```

Now that we understood the **working of function**, let's see the types of function in C++

Types of function

We have two types of function in C++:



- 1) Built-in functions
- 2) User-defined functions

1) Built-in functions

Built-in functions are also known as library functions. We need not to declare and define these functions as they are already written in the C++ libraries such as `iostream`, `cmath` etc. We can directly call them when we need.

Example: C++ built-in function example

Here we are using built-in function `pow(x,y)` which is x to the power y . This function is declared in `cmath` header file so we have included the file in our program using `#include` directive.

```
#include <iostream>
```

```
#include <cmath>
```

```
using namespace std;
```

```

int main(){

    /* Calling the built-in function

    * pow(x, y) which is x to the power y

    * We are directly calling this function

    */

    cout<<pow(2,5);

    return 0;

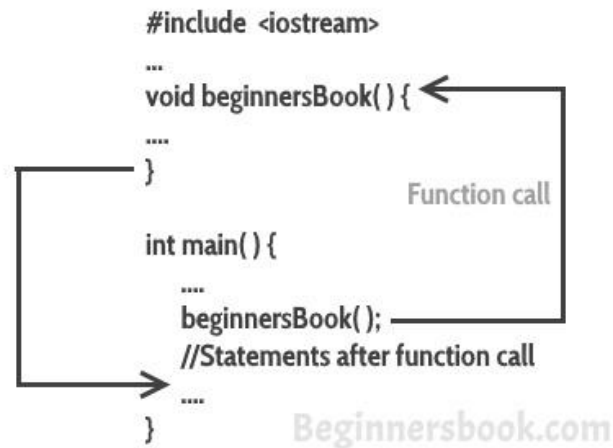
}

```

Output:

32

2) User-defined functions



We have already seen user-defined functions, the example we have given at the beginning of this is an example of user-defined function. The functions that we declare and write in our programs are user-defined functions. Let's see another example of user-defined functions.

User-defined functions

```

#include <iostream>

#include <cmath>

using namespace std;

//Declaring the function sum

int sum(int,int);

int main(){

    int x, y;

    cout<<"enter first number: ";

    cin>> x;

    cout<<"enter second number: ";

    cin>>y;

    cout<<"Sum of these two :"<<sum(x,y);

    return 0;

}

//Defining the function sum

int sum(int a, int b) {

    int c = a+b;

    return c;

}

```

Output:

enter first number: 22

enter second number: 19

Sum of these two : 41

Default Arguments in C++ Functions

The default arguments are used when you provide no arguments or only few arguments while calling a function. The default arguments are used during compilation of program. For example, let's say you have a user-defined function sum declared like this: `int sum(int a=10, int b=20)`, now while calling this function you do not provide any arguments, simply called `sum()`; then in this case the result would be 30, compiler used the default values 10 and 20 declared in function signature. If you pass only one argument like this: `sum(80)` then the result would be 100, using the passed argument 80 as first value and 20 taken from the default argument.

Example: Default arguments in C++

```
#include <iostream>

using namespace std;

int sum(int a, int b=10, int c=20);

int main(){

    /* In this case a value is passed as

    * 1 and b and c values are taken from

    * default arguments.

    */

    cout<<sum(1)<<endl;

    /* In this case a value is passed as

    * 1 and b value as 2, value of c values is

    * taken from default arguments.

    */
```

```

cout<<sum(1, 2)<<endl;

/* In this case all the three values are

* passed during function call, hence no

* default arguments have been used.

*/

cout<<sum(1, 2, 3)<<endl;

return 0;

}

int sum(int a, int b, int c){

    int z;

    z = a+b+c;

    return z;

}

```

Output:

31

23

6

Rules of default arguments

As you have seen in the above example that I have assigned the default values for only two arguments b and c during function declaration. It is up to you to assign default values to all arguments or only selected arguments but remember the following rule while assigning default values to only some of the arguments:

If you assign default value to an argument, the subsequent arguments must have default values assigned to them, else you will get compilation error.

For example: Let's see some valid and invalid cases.

Valid: Following function declarations are valid –

```
int sum(int a=10, int b=20, int c=30);
```

```
int sum(int a, int b=20, int c=30);
```

```
int sum(int a, int b, int c=30);
```

Invalid: Following function declarations are invalid –

```
/* Since a has default value assigned, all the
```

```
* arguments after a (in this case b and c) must have
```

```
* default values assigned
```

```
*/
```

```
int sum(int a=10, int b, int c=30);
```

```
/* Since b has default value assigned, all the
```

```
* arguments after b (in this case c) must have
```

```
* default values assigned
```

```
*/
```

```
int sum(int a, int b=20, int c);
```

```
/* Since a has default value assigned, all the
```

```
* arguments after a (in this case b and c) must have
```

```
* default values assigned, b has default value but
```

* c doesn't have, thats why this is also invalid

*/

```
int sum(int a=10, int b=20, int c);
```

C++ Recursion with example

The process in which a function calls itself is known as recursion and the corresponding function is called the **recursive function**. The popular example to understand the recursion is factorial function.

Factorial function: $f(n) = n * f(n-1)$, base condition: if $n \leq 1$ then $f(n) = 1$. Don't worry we will discuss what is base condition and why it is important.

In the following diagram. I have shown that how the factorial function is calling itself until the function reaches to the base condition.

Factorial function: $f(n) = n * f(n-1)$

Lets say we want to find out the factorial of 5 which means $n = 5$

$$f(5) = 5 * f(5-1) = 5 * f(4)$$



$$5 * 4 * f(4-1) = 20 * f(3)$$



$$20 * 3 * f(3-1) = 60 * f(2)$$



$$60 * 2 * f(2-1) = 120 * f(1)$$



$$120 * 1 * f(1-1) = 120 * f(0)$$



$$120 * 1 = 120$$

Let's solve the problem using C++ program.

C++ recursion example: Factorial

```
#include <iostream>
```

```
using namespace std;
```

```
//Factorial function
```

```
int f(int n){
```

```

/* This is called the base condition, it is
* very important to specify the base condition
* in recursion, otherwise your program will throw
* stack overflow error.

*/

if (n <= 1)

    return 1;

else

    return n*f(n-1);

}

int main(){

    int num;

    cout<<"Enter a number: ";

    cin>>num;

    cout<<"Factorial of entered number: "<<f(num);

    return 0;

}

```

Output:

Enter a number: 5

Factorial of entered number: 120

Base condition

In the above program, you can see that I have provided a base condition in the recursive function. The condition is:

```
if (n <= 1)

    return 1;
```

The purpose of recursion is to divide the problem into smaller problems till the base condition is reached. For example in the above factorial program I am solving the factorial function $f(n)$ by calling a smaller factorial function $f(n-1)$, this happens repeatedly until the n value reaches base condition($f(1)=1$). If you do not define the base condition in the recursive function then you will get stack overflow error.

Direct recursion vs indirect recursion

Direct recursion: When function calls itself, it is called direct recursion, the example we have seen above is a direct recursion example.

Indirect recursion: When function calls another function and that function calls the calling function, then this is called indirect recursion. For example: function A calls function B and Function B calls function A.

Indirect Recursion Example in C++

```
#include <iostream>

using namespace std;

int fa(int);

int fb(int);

int fa(int n){

    if(n<=1)

        return 1;
```



```
    else

        return n*fb(n-1);

}

int fb(int n){

    if(n<=1)

        return 1;

    else

        return n*fa(n-1);

}

int main(){

    int num=5;

    cout<<fa(num);

    return 0;

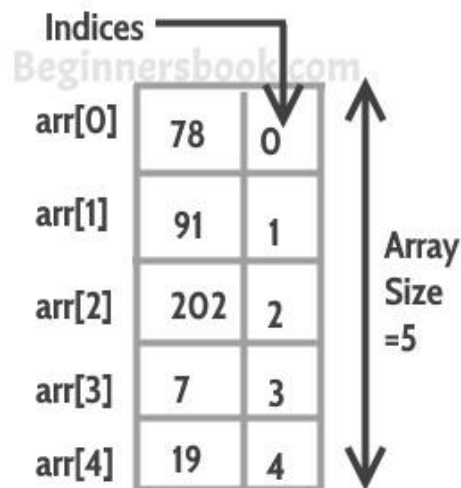
}
```

Output:

120

Arrays in C++

An array is a collection of similar items stored in contiguous memory locations. In programming, sometimes a simple variable is not enough to hold all the data. For example, let's say we want to store the marks of 500 students, having 500 different variables for this task is not feasible, we can define an array with size 500 that can hold the marks of all students.



The diagram illustrates an array structure. It features a table with five rows, each representing an element in the array. The first column of the table lists the indices: arr[0], arr[1], arr[2], arr[3], and arr[4]. The second column contains the values stored at these indices: 78, 91, 202, 7, and 19. A third column, labeled 'Indices' at the top, contains the corresponding index values: 0, 1, 2, 3, and 4. A vertical double-headed arrow on the right side of the table indicates the 'Array Size = 5'. A horizontal arrow points from the 'Indices' label to the index values in the third column.

	Indices	
arr[0]	78	0
arr[1]	91	1
arr[2]	202	2
arr[3]	7	3
arr[4]	19	4

Array Size = 5

Declaring an array in C++

There are couple of ways to declare an array.

Method 1:

```
int arr[5];
```

```
arr[0] = 10;
```

```
arr[1] = 20;
```

```
arr[2] = 30;
```

```
arr[3] = 40;
```

```
arr[4] = 50;
```

Method 2:

```
int arr[] = {10, 20, 30, 40, 50};
```

Method 3:

```
int arr[5] = {10, 20, 30, 40, 50};
```

Accessing Array Elements

Array index starts with 0, which means the first array element is at index 0, second is at index 1 and so on. We can use this information to display the array elements. See the code below:

```
#include <iostream>
```

```
using namespace std;
```

```
int main(){
```

```
    int arr[] = {11, 22, 33, 44, 55};
```

```
    cout<<arr[0]<<endl;
```

```
    cout<<arr[1]<<endl;
```

```
    cout<<arr[2]<<endl;
```

```
    cout<<arr[3]<<endl;
```

```
    cout<<arr[4]<<endl;
```

```
    return 0;
```

```
}
```

Output:

11

22

33

44

55

Although this code worked fine, displaying all the elements of array like this is not recommended. When you want to access a particular array element then this is fine but if you want to display all the elements then you should use a loop like this:

```
#include <iostream>
```

```
using namespace std;
```

```
int main(){
```

```
    int arr[] = {11, 22, 33, 44, 55};
```

```
    int n=0;
```

```
    while(n<=4){
```

```
        cout<<arr[n]<<endl;
```

```
        n++;
```

```
    }
```

```
    return 0;
```

```
}
```

Multidimensional Arrays in C++

Multidimensional arrays are also known as **array of arrays**. The data in multidimensional array is stored in a tabular form as shown in the diagram below:

	column 1	column 2	column 3	column 4	column 5
row1	arr[0][0]	arr[0][1]	arr[0][2]	arr[0][3]	arr[0][4]
row2	arr[1][0]	arr[1][1]	arr[1][2]	arr[1][3]	arr[1][4]
row3	arr[2][0]	arr[2][1]	arr[2][2]	arr[2][3]	arr[2][4]

A two dimensional array:

```
int arr[2][3];
```

This array has total $2*3 = 6$ elements.

A three dimensional array:

```
int arr[2][2][2];
```

This array has total $2*2*2 = 8$ elements.

Two dimensional array

Let's see how to declare, initialize and access Two Dimensional Array elements.

How to declare a two dimensional array?

```
int myarray[2][3];
```

Initialization:

We can initialize the array in many ways:

Method 1:

```
int arr[2][3] = {10, 11 ,12 ,20 ,21 , 22};
```

Method

2:

This way of initializing is preferred as you can visualize the rows and columns here.

```
int arr[2][3] = {{10, 11 ,12} , {20 ,21 , 22}};
```

Accessing		array	elements:
arr[0][0]	—	first	element
arr[0][1]	—	second	element
arr[0][2]	—	third	element
arr[1][0]	—	fourth	element
arr[1][1]	—	fifth	element
arr[1][2]	— sixth element		

Example: Two dimensional array in C++

```
#include <iostream>
```

```
using namespace std;
```

```
int main(){
```

```
    int arr[2][3] = {{11, 22, 33}, {44, 55, 66}};
```

```
    for(int i=0; i<2;i++){
```

```
        for(int j=0; j<3; j++){
```

```
            cout<<"arr["<<i<<"]["<<j<<"]: "<<arr[i][j]<<endl;
```

```
        }
```

```
    }
```

```
    return 0;
```

```
}
```

Output:

```
arr[0][0]: 11
```

```
arr[0][1]: 22
```

```
arr[0][2]: 33
```

```
arr[1][0]: 44
```

```
arr[1][1]: 55
```

```
arr[1][2]: 66
```

Three dimensional array

Lets see how to declare, initialize and access Three Dimensional Array elements.

Declaring a three dimensional array:

```
int myarray[2][3][2];
```

Initialization:

We can initialize the array in many ways:

Method 1:

```
int arr[2][3][2] = {1, -1 ,2 ,-2 , 3 , -3, 4, -4, 5, -5, 6, -6};
```

Method

2:

This way of initializing is preferred as you can visualize the rows and columns here.

```
int arr[2][3][2] = {  
    { {1,-1}, {2, -2}, {3, -3}},  
    { {4, -4}, {5, -5}, {6, -6}}  
}
```

Three dimensional array example

```
#include <iostream>

using namespace std;

int main(){

    // initializing the array

    int arr[2][3][2] = {

        { {1,-1}, {2,-2}, {3,-3} },

        { {4,-4}, {5,-5}, {6,-6} }

    };

    // displaying array values

    for (int x = 0; x < 2; x++) {

        for (int y = 0; y < 3; y++) {

            for (int z = 0; z < 2; z++) {

                cout<<arr[x][y][z]<<" ";

            }

        }

    }

    return 0;

}
```

Output:

1 -1 2 -2 3 -3 4 -4 5 -5 6 -6

Passing Array to Function in C++

You can pass array as an argument to a function just like you pass variables as arguments. In order to pass array to the function you just need to **mention the array name during function call** like this:

```
function_name(array_name);
```

Example: Passing arrays to a function

In this example, we are passing two arrays a & b to the function sum(). This function adds the corresponding elements of both the arrays and display them.

```
#include <iostream>
```

```
using namespace std;
```

```
/* This function adds the corresponding
```

```
* elements of both the arrays and
```

```
* displays it.
```

```
*/
```

```
void sum(int arr1[], int arr2[]){
```

```
    int temp[5];
```

```
    for(int i=0; i<5; i++){
```

```
        temp[i] = arr1[i]+arr2[i];
```

```
        cout<<temp[i]<<endl;
```

```
    }
```

```
}
```

```
int main(){
```

```
int a[5] = {10, 20, 30, 40 ,50};

int b[5] = {1, 2, 3, 4, 5};

//Passing arrays to function

sum(a, b);

return 0;

}
```

Output:

11

22

33

44

55

Example 2: Passing multidimensional array to function

In this example we are passing a multidimensional array to the function square which displays the square of each element.

```
#include <iostream>
```

```
#include <cmath>
```

```
using namespace std;
```

```
/* This method prints the square of each
```

```
* of the elements of multidimensional array
```

```
*/
```

```

void square(int arr[2][3]){

    int temp;

    for(int i=0; i<2; i++){

        for(int j=0; j<3; j++){

            temp = arr[i][j];

            cout<<pow(temp, 2)<<endl;

        }

    }

}

int main(){

    int arr[2][3] = {

        {1, 2, 3},

        {4, 5, 6}

    };

    square(arr);

    return 0;

}

```

Output:

1
4
9

16

25

36

Strings in C++

Strings are words that are made up of characters, hence they are known as sequence of characters. In C++ we have two ways to create and use strings: 1) By creating char arrays and treat them as string 2) By creating string object

Let's discuss these two ways of creating string first and then we will see which method is better and why.

1) Array of Characters – Also known as C Strings

Example 1:

A simple example where we have initialized the char array during declaration.

```
#include <iostream>

using namespace std;

int main(){

    char book[50] = "A Song of Ice and Fire";

    cout<<book;

    return 0;

}
```

Output:

A Song of Ice and Fire

Example 2: Getting user input as string

This can be considered as inefficient method of reading user input, why? Because when we read the user input string using cin then only the first word of the string is stored in char array and rest get ignored. The cin function considers the space in the string as delimiter and ignore the part after it.

```
#include <iostream>

using namespace std;

int main(){

    char book[50];

    cout<<"Enter your favorite book name:";

    //reading user input

    cin>>book;

    cout<<"You entered: "<<book;

    return 0;

}
```

Output:

Enter your favorite book name:The Murder of Roger Ackroyd

You entered: The

You can see that only the “The” got captured in the book and remaining part after space got ignored. How to deal with this then? Well, for this we can use `cin.get` function, which reads the complete line entered by user.

Example 3: Correct way of capturing user input string using `cin.get`

```
#include <iostream>

using namespace std;

int main(){

    char book[50];
```

```

cout<<"Enter your favorite book name:";

//reading user input

cin.get(book, 50);

cout<<"You entered: "<<book;

return 0;

}

```

Output:

Enter your favorite book name:The Murder of Roger Ackroyd

You entered: The Murder of Roger Ackroyd

Drawback of this method

- 1) Size of the char array is fixed, which means the size of the string created through it is fixed in size, more memory cannot be allocated to it during runtime. For example, lets say you have created an array of character with the size 10 and user enters the string of size 15 then the last five characters would be truncated from the string. On the other hand if you create a larger array to accommodate user input then the memory is wasted if the user input is small and array is much larger then what is needed.
- 2) In this method, you can only use the in-built functions created for array which don't help much in string manipulation.

What is the solution of these problems

We can create string using string object. Lets see how we can do it.

String object in C++

Till now we have seen how to handle strings in C++ using char arrays. Lets see another and better way of handling strings in C++ – string objects.

```

#include<iostream>

using namespace std;

int main(){

    // This is how we create string object

    string str;

    cout<<"Enter a String:";

    /* This is used to get the user input

    * and store it into str

    */

    getline(cin,str);

    cout<<"You entered: ";

    cout<<str<<endl;

    /* This function adds a character at

    * the end of the string

    */ str.push_back('A');

    cout<<"The string after push_back: "<<str<<endl;

    /* This function deletes a character from

    * the end of the string

    */

    str.pop_back();

    cout << "The string after pop_back: "<<str<<endl;

```



```
    return 0;  
  
}
```

Output:

Enter a String:XYZ

You entered: XYZ

The string after push_back: XYZA

The string after pop_back: XYZ

The advantage of using this method is that you need not to declare the size of the string, the size is determined at run time, so this is better memory management method. The memory is allocated dynamically at runtime so no memory is wasted.

Pointers in C++

Pointer is a variable in C++ that holds the address of another variable. They have data type just like variables, for example an integer type pointer can hold the address of an integer variable and an character type pointer can hold the address of char variable.

Syntax of pointer

```
data_type *pointer_name;
```

How to declare a pointer

```
/* This pointer p can hold the address of an integer
```

```
* variable, here p is a pointer and var is just a
```

```
* simple integer variable
```

```
*/
```

```
int *p, var
```

Assignment

As I mentioned above, an integer type pointer can hold the address of another int variable. Here we have an integer variable var and pointer p holds the address of var. To assign the address of variable to pointer we use **ampersand symbol (&)**.

```
/* This is how you assign the address of another variable
```

```
* to the pointer
```

```
*/
```

```
p = &var;
```

How to use it

```
// This will print the address of variable var
```

```

cout<<&var;

/* This will also print the address of variable
 * var because the pointer p holds the address of var
 */

cout<<p;

/* This will print the value of var, This is
 * important, this is how we access the value of
 * variable through pointer
 */

cout<<*p;

```

Example of Pointer

Lets take a simple example to understand what we discussed above.

```

#include <iostream>

using namespace std;

int main(){

    //Pointer declaration

    int *p, var=101;

    //Assignment

    p = &var;

    cout<<"Address of var: "<<&var<<endl;

    cout<<"Address of var: "<<p<<endl;

```

```

cout<<"Address of p: "<<&p<<endl;

cout<<"Value of var: "<<*p;

return 0;

}

```

Output:

Address of var: 0x7fff5dffc0c

Address of var: 0x7fff5dffc0c

Address of p: 0x7fff5dffc10

Value of var: 101

Pointer and arrays

While handling arrays with pointers you need to take care few things. First and very important point to note regarding arrays is that the array name alone represents the base address of array so while assigning the address of array to pointer don't use ampersand sign(&). Do it like this:

Correct: Because arr represent the address of array.

```
p = arr;
```

Incorrect:

```
p = &arr;
```

Example: Traversing the array using Pointers

```
#include <iostream>
```

```
using namespace std;
```

```
int main(){
```

```

//Pointer declaration

int *p;

//Array declaration

int arr[]={1, 2, 3, 4, 5, 6};

//Assignment

p = arr;

for(int i=0; i<6;i++){

    cout<<*p<<endl;

    //++ moves the pointer to next int position

    p++;

}

return 0;

}

```

Output:

```

1
2
3
4
5
6

```

How to increment pointer address and pointer's value

When we are accessing the value of a variable through pointer, sometimes we just need to increment or decrement the value of variable though it or we may need to move the pointer to next int position(just like we did above while working with arrays). The++ operator is used for this purpose. One of the example of ++ operator we have seen above where we traversed the array using pointer by incrementing the pointer value using ++ operator. Let's see few more cases.

```
// Pointer moves to the next int position (as if it was an array)
```

```
p++;
```

```
// Pointer moves to the next int position (as if it was an array)
```

```
++p;
```

```
/* All the following three cases are same they increment the value
```

```
 * of variable that the pointer p points.
```

```
*/
```

```
++*p;
```

```
++(*p);
```

```
++*(p);
```

C++ 'this' Pointer

The **this** pointer holds the address of current object, in simple words you can say that this pointer points to the current object of the class. Let's take an example to understand this concept.

C++ Example: this pointer

Here you can see that we have two data members num and ch. In member function setMyValues() we have two local variables having same name as data members name. In such case if you want to assign the local variable value to the data members then you won't be able to do until unless you use this pointer, because the compiler won't know that you are referring to object's data members unless you use this pointer. This is one of the example where you must use **this** pointer.

```
#include <iostream>

using namespace std;

class Demo {

private:

    int num;

    char ch;

public:

    void setMyValues(int num, char ch){

        this->num =num;

        this->ch=ch;

    }

    void displayMyValues(){
```

```

    cout<<num<<endl;

    cout<<ch;

}

};

int main(){

    Demo obj;

    obj.setMyValues(100, 'A');

    obj.displayMyValues();

    return 0;

}

```

Output:

100

A

Example 2: function chaining calls using this pointer

Another example of using this pointer is to return the reference of current object so that you can chain function calls, this way you can call all the functions for the current object in one go. Another important point to note in this program is that I have incremented the value of object's num in the second function and you can see in the output that it actually incremented the value that we have set in the first function call. This shows that the chaining is sequential and the changes made to the object's data members retains for further chaining calls.

```

#include <iostream>

using namespace std;

```



```

class Demo {

private:

    int num;

    char ch;

public:

    Demo &setNum(int num){

        this->num =num;

        return *this;

    }

    Demo &setCh(char ch){

        this->num++;

        this->ch =ch;

        return *this;

    }

    void displayMyValues(){

        cout<<num<<endl;

        cout<<ch;

    }

};

int main(){

    Demo obj;

```

```
//Chaining calls  
  
obj.setNum(100).setCh('A');  
  
obj.displayMyValues();  
  
return 0;  
  
}
```

Output:

101

A

OOPs Concepts in C++

Object oriented programming is a way of solving complex problems by breaking them into smaller problems using objects. Before Object Oriented Programming (commonly referred as OOP), programs were written in procedural language, they were nothing but a long list of instructions. On the other hand, the OOP is all about creating objects that can interact with each other, this makes it easier to develop programs in OOP as we can understand the relationship between them.

Object Oriented Programming (OOP)

In Object oriented programming we write programs using classes and objects utilising features of OOPs such as **abstraction**, **encapsulation**, **inheritance** and **polymorphism**

Class and Objects

A class is like a blueprint of data member and functions and object is an instance of class. For example, lets say we have a class **Car** which has data members (variables) such as speed, weight, price and functions such as gearChange(), slowDown(), brake() etc. Now lets say I create a object of this class named FordFigo which uses these data members and functions and give them its own values. Similarly we can create as many objects as we want using the blueprint(class).

```
//Class name is Car
```

```
class Car
```

```
{
```

```
    //Data members
```

```
    char name[20];
```

```
    int speed;
```

```
    int weight;
```

public:

```
//Functions
```

```
void brake(){
```

```
}
```

```
void slowDown(){
```

```
}
```

```
};
```

```
int main()
```

```
{
```

```
//ford is an object
```

```
Car ford;
```

```
}
```

Abstraction

Abstraction is a process of hiding irrelevant details from user. For example, When you send an sms you just type the message, select the contact and click send, the phone shows you that the message has been sent, what actually happens in background when you click send is hidden from you as it is not relevant to you.

Encapsulation

Encapsulation is a process of combining data and function into a single unit like capsule. This is to avoid the access of private data members from outside the class. To achieve encapsulation, we make all data members of class private and create public functions,

using them we can get the values from these data members or set the value to these data members.

Inheritance

Inheritance is a feature using which an object of child class acquires the properties of parent class.

```
#include <iostream>

using namespace std;

class ParentClass {

    //data member

    public:

        int var1 =100;

};

class ChildClass: public ParentClass {

    public:

        int var2 = 500;

};

int main(void) {

    ChildClass obj;

}
```

Now this object obj can use the properties (such as variable var1) of ParentClass.

Polymorphism

Function overloading and Operator overloading are examples of polymorphism. Polymorphism is a feature using which an object behaves differently in different situation. In function overloading we can have more than one function with same name but different numbers, type or sequence of arguments.

Polymorphism Example

```
#include <iostream>

using namespace std;

class Sum {

public:

    int add(int num1,int num2){

        return num1 + num2;

    }

    int add(int num1, int num2, int num3){

        return num1 + num2 + num3;

    }

};

int main(void) {

    //Object of class Sum

    Sum obj;

    //This will call the second add function

    cout<<obj.add(10, 20, 30)<<endl;

    //This will call the first add function
```

```
    cout<<obj.add(11, 22);  
  
    return 0;  
  
}
```

Output:

60

33

Constructors in C++

Constructor is a special member function of a class that initializes the object of the class. Constructor name is same as class name and it doesn't have a return type. Lets take a simple example to understand the working of constructor.

Simple Example: How to use constructor in C++

Read the comments in the following program to understand each part of the program.

```
#include <iostream>

using namespace std;

class constructorDemo{

public:

    int num;

    char ch;

    /* This is a default constructor of the
    * class, do note that it's name is same as
    * class name and it doesn't have return type.
    */

    constructorDemo() {

        num = 100; ch = 'A';

    }

};

int main(){
```



```

/* This is how we create the object of class,

* I have given the object name as obj, you can

* give any name, just remember the syntax:

* class_name object_name;

*/

constructorDemo obj;

/* This is how we access data members using object

* we are just checking that the value we have

* initialized in constructor are reflecting or not.

*/

cout<<"num: "<<obj.num<<endl;

cout<<"ch: "<<obj.ch;

return 0;

}

```

Output:

num: 100

ch: A

Constructor vs Member function

Now that we know what is constructor, lets discuss how a constructor is different from member function of the class.

- 1) Constructor doesn't have a return type. Member function has a return type.
 - 2) Constructor is automatically called when we create the object of the class. Member function needs to be called explicitly using object of class.
 - 3) When we do not create any constructor in our class, C++ compiler generates a default constructor and insert it into our code. The same does not apply to member functions.
- This is how a compiler generated default constructor looks:

```
class XYZ  
  
{  
  
    ....  
  
    XYZ()  
  
    {  
  
        //Empty no code  
  
    }  
  
};
```

Types of Constructor in C++

There are two types of constructor in C++. 1) Default constructor 2) Parameterized constructor

1) Default Constructor

A default constructor doesn't have any arguments (or parameters)

```
#include <iostream>  
  
using namespace std;  
  
class Website{  
  
public:
```

```

//Default constructor

Website() {

    cout<<"Welcome to BeginnersBook"<<endl;

}

};

int main(void){

    /*creating two objects of class Website.

    * This means that the default constructor

    * should have been invoked twice.

    */

    Website obj1;

    Website obj2;

    return 0;

}

```

Output:

Welcome to BeginnersBook

Welcome to BeginnersBook

When you don't specify any constructor in the class, a default constructor with no code (empty body) would be inserted into your code by compiler.

2) Parameterized Constructor

Constructors with parameters are known as Parameterized constructors. These type of constructor allows us to pass arguments while object creation. Let's see how they look:

Lets say class name is XYZ

Default constructor:

```
XYZ() {  
  
}
```

....

```
XYZ obj;
```

....

Parameterized Constructor:

```
XYZ(int a, int b) {  
  
  
}
```

...

```
XYZ obj(10, 20);
```

Example:

```
#include <iostream>
```

```
using namespace std;
```

```
class Add{
```

```
public:
```

```
    //Parameterized constructor
```

```

Add(int num1, int num2) {

    cout<<(num1+num2)<<endl;

}

};

int main(void){

    /* One way of creating object. Also

    * known as implicit call to the

    * constructor

    */

    Add obj1(10, 20);

    /* Another way of creating object. This

    * is known as explicit calling the

    * constructor.

    */

    Add obj2 = Add(50, 60);

    return 0;

}

```

Output:

30

110

destructors in C++

A destructor is a special member function that works just opposite to constructor, unlike constructors that are used for initializing an object, destructors destroy (or delete) the object.

Syntax of Destructor

```
~class_name()  
  
{  
  
    //Some code  
  
}
```

Similar to constructor, the destructor name should exactly match with the class name. A destructor declaration should always begin with the tilde(~) symbol as shown in the syntax above.

When does the destructor get called

A destructor is **automatically called** when:

- 1) The program finished execution.
- 2) When a scope (the { } parenthesis) containing local variable ends.
- 3) When you call the delete operator.

Destructor Example

```
#include <iostream>  
  
using namespace std;  
  
class HelloWorld{  
  
public:
```

```

//Constructor

HelloWorld(){

    cout<<"Constructor is called"<<endl;

}

//Destructor

~HelloWorld(){

    cout<<"Destructor is called"<<endl;

}

//Member function

void display(){

    cout<<"Hello World!"<<endl;

}

};

int main(){

    //Object created

    HelloWorld obj;

    //Member function called

    obj.display();

    return 0;

}

```

Output:

Constructor is called

Hello World!

Destructor is called

Destructor rules

- 1) Name should begin with tilde sign(~) and must match class name.
- 2) There cannot be more than one destructor in a class.
- 3) Unlike constructors that can have parameters, destructors do not allow any parameter.
- 4) They do not have any return type, just like constructors.
- 5) When you do not specify any destructor in a class, compiler generates a default destructor and inserts it into your code.

Structures in C++

Structure is a compound data type that contains different variables of different types. For example, you want to store Student details like student name, student roll num, student age. You have two ways to do it, one way is to create different variables for each data, but the downfall of this approach is that if you want to store the details of multiple students, in that case it is not feasible to create separate set of variables for each student. The second and best way of doing it by creating a structure like this:

```
struct Student  
  
{  
  
    char stuName[30];  
  
    int stuRollNo;  
  
    int stuAge;  
  
};
```

Now these three members combined will act like a separate variable and you can create structure variable like this:

```
structure_name variable_name
```

So if you want to hold the information of two students using this structure then you can do it like this:

```
Student s1, s2;
```

Then I can access the members of Student structure like this:

```
//Assigning name to first student
```

```
s1.stuName = "Ajeet";
```

```
//Assigning age to the second student
```

```
s2.stuAddr = 22;
```

Similarly I can set and get the values of other data members of the structure for every student. Lets see a complete example to put this up all together:

Structure Example in C++

```
#include <iostream>

using namespace std;

struct Student{

    char stuName[30];

    int stuRollNo;

    int stuAge;

};

int main(){

    Student s;

    cout<<"Enter Student Name: ";

    cin.getline(s.stuName, 30);

    cout<<"ENter Student Roll No: ";

    cin>>s.stuRollNo;

    cout<<"Enter Student Age: ";

    cin>>s.stuAge;

    cout<<"Student Record:"<<endl;

    cout<<"Name: "<<s.stuName<<endl;
```

```
cout<<"Roll No: "<<s.stuRollNo<<endl;

cout<<"Age: "<<s.stuAge;

return 0;

}
```

Output:

Enter Student Name: Negan

ENter Student Roll No: 4101003

Enter Student Age: 22

Student Record:

Name: Negan

Roll No: 4101003

Age: 22

Structure and Function in C++

We learnt about structures, the compound data type that groups different types of variables. We will learn how to pass structures as an argument to the function and how to return the structure from the function.

How to pass structure as an argument to function

Here we have a function `printStudentInfo()` which takes structure `Student` as an argument and prints the details of student using structure variable. The important point to note here is that you should always declare the structure before function declarations, otherwise you will get compilation error.

```
#include <iostream>

using namespace std;

struct Student{

    char stuName[30];

    int stuRollNo;

    int stuAge;

};

void printStudentInfo(Student);

int main(){

    Student s;

    cout<<"Enter Student Name: ";

    cin.getline(s.stuName, 30);

    cout<<"Enter Student Roll No: ";
```

```

    cin>>s.stuRollNo;

    cout<<"Enter Student Age: ";

    cin>>s.stuAge;

    printStudentInfo(s);

    return 0;

}

void printStudentInfo(Student s){

    cout<<"Student Record:"<<endl;

    cout<<"Name: "<<s.stuName<<endl;

    cout<<"Roll No: "<<s.stuRollNo<<endl;

    cout<<"Age: "<<s.stuAge;

}

```

Output:

Enter Student Name: Rick

Enter Student Roll No: 666123

Enter Student Age: 19

Student Record:

Name: Rick

Roll No: 666123

Age: 19

How to return the Structure from a Function

In this example we have two functions one gets the values from user, assign them to structure members and returns the structure and the other function takes that structure as argument and print the details.

```
#include <iostream>

using namespace std;

struct Student{

    char stuName[30];

    int stuRollNo;

    int stuAge;

};

Student getStudentInfo();

void printStudentInfo(Student);

int main(){

    Student s;

    s = getStudentInfo();

    printStudentInfo(s);

    return 0;

}

/* This function prompt the user to input student

* details, stores them in structure members

* and returns the structure
```

*/

```
Student getStudentInfo(){  
    Student s;  
  
    cout<<"Enter Student Name: ";  
  
    cin.getline(s.stuName, 30);  
  
    cout<<"Enter Student Roll No: ";  
  
    cin>>s.stuRollNo;  
  
    cout<<"Enter Student Age: ";  
  
    cin>>s.stuAge;  
  
    return s;  
}  
  
void printStudentInfo(Student s){  
    cout<<"Student Record:"<<endl;  
  
    cout<<"Name: "<<s.stuName<<endl;  
  
    cout<<"Roll No: "<<s.stuRollNo<<endl;  
  
    cout<<"Age: "<<s.stuAge;  
  
}
```

Output:

Enter Student Name: Tyrion lannister

Enter Student Roll No: 333901

Enter Student Age: 39

Student Record:

Name: Tyrion lannister

Roll No: 333901

Age: 39

Enumeration in C++

Enum is a user defined data type where we specify a set of values for a variable and the variable can only take one out of a small set of possible values. We use enum keyword to define a Enumeration.

```
enum direction {East, West, North, South}dir;
```

Here Enumeration name is direction which can only take one of the four specified values, the dir at the end of the declaration is an enum variable.

Let's take a simple example to understand this:

Here I have assigned the value West to the enum variable dir and when I displayed the value of dir it shows 1. This is because by default the values are in increasing order starting from 0, which means East is 0, West is 1, North is 2 and South is 3.

Simple enum Example

```
#include<iostream>

using namespace std;

enum direction {East, West, North, South}dir;

int main()

{

    dir = West;

    cout<<dir;

    return 0;

}
```

Another way to declare enum variable

As we have seen in the above example that I have declared the enum variable dir during enum declaration, there is another way to declare an enum variable.

```
#include <iostream>

using namespace std;

enum direction {East, West, North, South};

int main(){

    direction dir;

    dir = South;

    cout<<dir;

    return 0;

}
```

Output:

3

Why use enum in C++

Now that we understand what is enum and how to use them in program, let's discuss why we use them:

Enums are used only when we expect the variable to have one of the possible set of values, for example, we have a dir variable that holds the direction. Since we have four directions, this variable can take any one of the four values, if we try to assign a another random value to this variable, it will throw a compilation error. This increases compile-time checking and avoid errors that occurs by passing in invalid constants.

Another important place where they are used frequently are switch case statements, where all the values that case blocks expect can be defined in an enum. This way we can ensure

that the enum variable that we pass in switch parenthesis is not taking any random value that it shouldn't accept.

How to change default values of Enum

```
#include <iostream>

using namespace std;

enum direction {East=11, West=22, North=33, South=44};

int main(){

    direction dir;

    dir = South;

    cout<<dir;

    return 0;

}
```

Output:

44

Inheritance in C++

Inheritance is one of the features of Object Oriented Programming System (OOPs), it allows the child class to acquire the properties (the data members) and functionality (the member functions) of parent class.

What is child class

A class that inherits another class is known as child class, it is also known as derived class or subclass.

What is parent class

The class that is being inherited by other class is known as parent class, super class or base class.

Syntax of Inheritance

```
class parent_class
{
    //Body of parent class
};

class child_class : access_modifier parent_class
{
    //Body of child class
};
```

What are the advantages of using inheritance in C++ Programming

The main advantages of inheritance are **code reusability** and **readability**. When child class inherits the properties and functionality of parent class, we need not to write the same

code again in child class. This makes it easier to reuse the code, makes us write the less code and the code becomes much more readable.

Let's take a **real life example** to understand this: Lets assume that Human is a class that has properties such as height, weight, colour etc and functionality such as eating(), sleeping(), dreaming(), working() etc.

Now we want to create Male and Female class, these classes are different but since both Male and Female are humans they share some common properties and behaviors (functionality) so they can inherit those properties and functionality from Human class and rest can be written in their class separately.

This approach makes us write less code as both the classes inherited several properties and functions from base class thus we didn't need to re-write them. Also, this makes it easier to read the code.

Inheritance Example

Before we discuss the types of inheritance, lets take an example: Here we have two classes Teacher and MathTeacher, the MathTeacher class inherits the Teacher class which means Teacher is a parent class and MathTeacher is a child class. The child class can use the property collegeName of parent class.

Another important point to note is that when we create the object of child class it calls the constructor of child class and child class constructor automatically calls the constructor of base class.

```
#include <iostream>
```

```
using namespace std;
```

```
class Teacher {
```

```
public:
```

```
    Teacher(){
```

```

    cout<<"Hey Guys, I am a teacher"<<endl;

}

string collegeName = "Beginnersbook";

};

//This class inherits Teacher class

class MathTeacher: public Teacher {

public:

    MathTeacher(){

        cout<<"I am a Math Teacher"<<endl;

    }

    string mainSub = "Math";

    string name = "Negan";

};

int main() {

    MathTeacher obj;

    cout<<"Name: "<<obj.name<<endl;

    cout<<"College Name: "<<obj.collegeName<<endl;

    cout<<"Main Subject: "<<obj.mainSub<<endl;

    return 0;

}

```

Output:

Hey Guys, I am a teacher

I am a Math Teacher

Name: Negan

College Name: Beginnersbook

Main Subject: Math

Types of Inheritance in C++

- 1) Single inheritance
- 2) Multilevel inheritance
- 3) Multiple inheritance
- 4) Hierarchical inheritance
- 5) Hybrid inheritance

Single inheritance

In Single inheritance one class inherits one class exactly.

For example: Let's say we have class A and B

B inherits A

Example of Single inheritance:

```
#include <iostream>

using namespace std;

class A {

public:

    A(){
```

```

    cout<<"Constructor of A class"<<endl;

}

};

class B: public A {

public:

    B(){

        cout<<"Constructor of B class";

    }

};

int main() {

    //Creating object of class B

    B obj;

    return 0;

}

```

Output:

Constructor of A class

Constructor of B class

2)Multilevel Inheritance

In this type of inheritance one class inherits another child class.

C inherits B and B inherits A

Example of Multilevel inheritance:


```
#include <iostream>

using namespace std;

class A {

public:

    A(){

        cout<<"Constructor of A class"<<endl;

    }

};

class B: public A {

public:

    B(){

        cout<<"Constructor of B class"<<endl;

    }

};

class C: public B {

public:

    C(){

        cout<<"Constructor of C class"<<endl;

    }

};

int main() {
```

```
//Creating object of class C

C obj;

return 0;

}
```

Output:

Constructor of A class

Constructor of B class

Constructor of C class

Multiple Inheritance

In multiple inheritance, a class can inherit more than one class. This means that in this type of inheritance a single child class can have multiple parent classes. For example:

C inherits A and B both

Example of Multiple Inheritance:

```
#include <iostream>

using namespace std;

class A {

public:

    A(){

        cout<<"Constructor of A class"<<endl;

    }
```

```

};

class B {

public:

    B(){

        cout<<"Constructor of B class"<<endl;

    }

};

class C: public A, public B {

public:

    C(){

        cout<<"Constructor of C class"<<endl;

    }

};

int main() {

    //Creating object of class C

    C obj;

    return 0;

}

```

Constructor of A class

Constructor of B class

Constructor of C class

4) Hierarchical Inheritance

In this type of inheritance, one parent class has more than one child class. For example:

Class B and C inherits class A

Example of Hierarchical inheritance:

```
#include <iostream>
```

```
using namespace std;
```

```
class A {
```

```
public:
```

```
    A(){
```

```
        cout<<"Constructor of A class"<<endl;
```

```
    }
```

```
};
```

```
class B: public A {
```

```
public:
```

```
    B(){
```

```
        cout<<"Constructor of B class"<<endl;
```

```
    }
```

```
};
```

```
class C: public A{
```

```
public:
```

```
    C(){
```

```
        cout<<"Constructor of C class"<<endl;

    }

};

int main() {

    //Creating object of class C

    C obj;

    return 0;

}
```

Output:

Constructor of A class

Constructor of C class

5) Hybrid Inheritance

Hybrid inheritance is a combination of more than one type of inheritance. For example, A child and parent class relationship that follows multiple and hierarchical inheritance both can be called hybrid inheritance.

Polymorphism in C++

Polymorphism is a feature of OOPs that allows the object to behave differently in different conditions. In C++ we have two types of polymorphism:

- 1) Compile time Polymorphism – This is also known as static (or early) binding.
- 2) Runtime Polymorphism – This is also known as dynamic (or late) binding.

1) Compile time Polymorphism

Function overloading and Operator overloading are perfect example of Compile time polymorphism.

Compile time Polymorphism Example

In this example, we have two functions with same name but different number of arguments. Based on how many parameters we pass during function call determines which function is to be called, this is why it is considered as an example of polymorphism because in different conditions the output is different. Since, the call is determined during compile time that's why it is called compile time polymorphism.

```
#include <iostream>

using namespace std;

class Add {

public:

    int sum(int num1, int num2){

        return num1+num2;

    }

    int sum(int num1, int num2, int num3){

        return num1+num2+num3;
```

```

    }

};

int main() {

    Add obj;

    //This will call the first function

    cout<<"Output: "<<obj.sum(10, 20)<<endl;

    //This will call the second function

    cout<<"Output: "<<obj.sum(11, 22, 33);

    return 0;

}

```

Output:

Output: 30

Output: 66

2) Runtime Polymorphism

Function overriding is an example of Runtime polymorphism.

Function Overriding: When child class declares a method, which is already present in the parent class then this is called function overriding, here child class overrides the parent class.

In case of function overriding we have two definitions of the same function, one is parent class and one in child class. The call to the function is determined at **runtime** to decide

which definition of the function is to be called, that's the reason it is called runtime polymorphism.

Example of Runtime Polymorphism

```
#include <iostream>

using namespace std;

class A {

public:

    void disp(){

        cout<<"Super Class Function"<<endl;

    }

};

class B: public A{

public:

    void disp(){

        cout<<"Sub Class Function";

    }

};

int main() {

    //Parent class object

    A obj;

    obj.disp();
```



```
//Child class object  
  
B obj2;  
  
obj2.disp();  
  
return 0;  
  
}
```

Output:

Super Class Function

Sub Class Function

Function overloading in C++

Function overloading is a C++ Programming feature that allows us to have more than one function having same name but different parameter list, when I say parameter list, it means the data type and sequence of the parameters, for example the parameters list of a function `myfuncn(int a, float b)` is `(int, float)` which is different from the function `myfuncn(float a, int b)` parameter list `(float, int)`. Function overloading is a compile-time polymorphism.

Now that we know what is parameter list, let's see the rules of overloading: we can have following functions in the same scope.

```
sum(int num1, int num2)
```

```
sum(int num1, int num2, int num3)
```

```
sum(int num1, double num2)
```

The easiest way to remember this rule is that the parameters should qualify any one or more of the following conditions, they should have different **type**, **number** or **sequence** of parameters.

For example:

These two functions have different parameter **type**:

```
sum(int num1, int num2)
```

```
sum(double num1, double num2)
```

These two have different **number** of parameters:

```
sum(int num1, int num2)
```

```
sum(int num1, int num2, int num3)
```

These two have different **sequence** of parameters:

```
sum(int num1, double num2)
```

```
sum(double num1, int num2)
```

All of the above three cases are valid case of overloading. We can have any number of functions, just remember that the parameter list should be different. For example:

```
int sum(int, int)
```

```
double sum(int, int)
```

This is not allowed as the parameter list is same. Even though they have different return types, its not valid.

Function overloading Example

Let's take an example to understand function overloading in C++.

```
#include <iostream>
```

```
using namespace std;
```

```
class Addition {
```

```
public:
```

```
    int sum(int num1,int num2) {
```

```
        return num1+num2;
```

```
    }
```

```
    int sum(int num1,int num2, int num3) {
```

```
        return num1+num2+num3;
```

```
    }
```

```
};
```

```

int main(void) {

    Addition obj;

    cout<<obj.sum(20, 15)<<endl;

    cout<<obj.sum(81, 100, 10);

    return 0;

}

```

Output:

35

191

Function overloading Example 2

As I mentioned in the beginning of this, that functions having different return types and same parameter list cannot be overloaded. However if the functions have different parameter list then they can have same or different return types to be eligible for overloading. In short the return type of a function does not play any role in function overloading. All that matters is the parameter list of function.

```

#include <iostream>

using namespace std;

class DemoClass {

public:

    int demoFunction(int i) {

        return i;
    }
}

```

```

    }

    double demoFunction(double d) {

        return d;

    }

};

int main(void) {

    DemoClass obj;

    cout<<obj.demoFunction(100)<<endl;

    cout<<obj.demoFunction(5005.516);

    return 0;

}

```

Output:

100

5006.52

Advantages of Function overloading

The main advantage of function overloading is to improve the **code readability** and allows **code reusability**. In the example 1, we have seen how we were able to have more than one function for the same task (addition) with different parameters, this allowed us to add two integer numbers as well as three integer numbers, if we wanted we could have some more functions with same name and four or five arguments. Imagine if we didn't have function overloading, we either have the limitation to add only two integers or we had to write different name functions for the same task addition, this would reduce the code readability and reusability.

Function Overriding in C++

Function overriding is a feature that allows us to have a same function in child class which is already present in the parent class. A child class inherits the data members and member functions of parent class, but when you want to override a functionality in the child class then you can use function overriding. It is like creating a new version of an old function, in the child class.

Function Overriding Example

To override a function you must have the same signature in child class. By signature I mean the data type and sequence of parameters. Here we don't have any parameter in the parent function so we didn't use any parameter in the child function.

```
#include <iostream>

using namespace std;

class BaseClass {
public:
    void disp(){
        cout<<"Function of Parent Class";
    }
};

class DerivedClass: public BaseClass{
public:
    void disp() {
        cout<<"Function of Child Class";
    }
}
```

```
};

int main() {

    DerivedClass obj = DerivedClass();

    obj.disp();

    return 0;

}
```

Output:

Function of Child Class

Note: In function overriding, the function in parent class is called the overridden function and function in child class is called overriding function.

How to call overridden function from the child class

As we have seen above that when we make the call to function (involved in overriding), the child class function (overriding function) gets called. What if you want to call the overridden function by using the object of child class. You can do that by creating the child class object in such a way that the reference of parent class points to it. Let's take an example to understand it.

```
#include <iostream>

using namespace std;

class BaseClass {

public:

    void disp(){

        cout<<"Function of Parent Class";
```

```

    }

};

class DerivedClass: public BaseClass{

public:

    void disp() {

        cout<<"Function of Child Class";

    }

};

int main() {

    /* Reference of base class pointing to

    * the object of child class.

    */

    BaseClass obj = DerivedClass();

    obj.disp();

    return 0;

}

```

Output:

Function of Parent Class

If you want to call the Overridden function from overriding function then you can do it like this:

```
parent_class_name::function_name
```


To do this in the above example, we can write following statement in the disp() function of child class:

```
BaseClass::disp();
```